Track Malfunctions in Flatland

TRailMix: Baruch Shteken[1] and Lena Friedek[2]

[1] Universität Potsdam, baruch.shteken@uni-potsdam.de
[2] Universität Potsdam, lena.friedek@uni-potsdam.de

Abstract. The abstract should briefly summarize the contents of the paper in 150–250 words.

Keywords: ASP · Flatland · MAPF · Rescheduling · VRSP.

1 Example Section

1.1 A Subsection Sample

Please note that the first paragraph of a section or subsection is not indented. The first paragraph that follows a table, figure, equation etc. does not need an indent, either.

Subsequent paragraphs, however, are indented.

```
1
2  {position(ID, POS, T', D)} :- position(ID, POS, T, D),
3  time(T), time(T'), T'= T+1, calculate(ID).
4
5  % end train when it reaches its destination
6  finish(ID, T) :- position(ID,(X,Y),T,D), end(ID,(X,Y),T'),
7  calculate(ID).
8
```

Here is a caption.

Sample Heading (Third Level) Only two levels of headings should be numbered. Lower level headings remain unnumbered; they are formatted as run-in headings.

```
1
2  import numpy as np
3
4  def incmatrix(genl1,genl2):
5  m = len(genl1)
6  n = len(genl2)
7  M = None #to become the incidence matrix
8  VT = np.zeros((n*m,1), int)  #dummy variable
9
```

Here you enter a caption.

Sample Heading (Fourth Level) The contribution should contain no more than four levels of headings. Table 1 gives a summary of all heading levels.

Table 1: Table captions should be placed above the tables.

Heading level	Example	Font size and style
Title (centered)	**Lecture Notes**	14 point, bold
1st-level heading	**1 Introduction**	12 point, bold
2nd-level heading	**2.1 Printing Area**	10 point, bold
3rd-level heading	**Run-in Heading in Bold.** Text follows	10 point, bold
4th-level heading	*Lowest Level Heading.* Text follows	10 point, italic

Displayed equations are centered and set on a separate line.

$$x + y = z \tag{1}$$

Please try to avoid rasterized images for line-art diagrams and schemas. Whenever possible, use vector graphics instead (see Fig. 1).

Fig. 1: A figure caption is always placed below the illustration. Please note that short captions are centered, while long ones are justified by the macro package automatically.

Theorem 1. *This is a sample theorem. The run-in heading is set in bold, while the following text appears in italics. Definitions, lemmas, propositions, and corollaries are styled the same way.*

Proof. Proofs, examples, and remarks have the initial word in italics, while the following text appears in normal font.

For citations of references, we prefer the use of square brackets and consecutive numbers. Citations using labels or the author/year convention are also acceptable. The following bibliography provides a sample reference list with entries for journal articles [?], an LNCS chapter [?], a book [?], proceedings without editors [?], and a homepage [?]. Multiple citations are grouped [?,?,?], [?,?,?,?].

Acknowledgments. A bold run-in heading in small font size at the end of the paper is used for general acknowledgments, for example: This study was funded by X (grant number Y).

Disclosure of Interests. It is now necessary to declare any competing interests or to specifically state that the authors have no competing interests. Please place the statement with a bold run-in heading in small font size beneath the (optional) acknowledgments[3], for example: The authors have no competing interests to declare that are relevant to the content of this article. Or: Author A has received research grants from Company W. Author B has received a speaker honorarium from Company X and owns stock in Company Y. Author C is a member of committee Z.

2 Introduction to Flatland and its malfunction handling

Flatland is a framework developed by AICrowd and SBB for multi-agent path finding problems and more specifically railway scheduling and rescheduling issues [5]. The initial simulation environment was designed for Python-based reinforcement learning approaches. It offers a contained experimentation sandbox around the vehicle rescheduling problem (VRSP) and simple two-dimensional visualisation of environments and agent movement.

2.1 Flatland and Answer Set Programming

Murphy has built a toolkit to integrate answer set programming solutions in the Flatland framework [6]. The approach solves instances with clingo [3] and then translates the solution to execute and visualize in Flatland. Environments are generated beforehand to adhere to Flatland prerequisites but also encodes the instance for clingo. The ASP encoding has to generate action predicates which communicate agent's steps to Flatland.

2.2 Malfunctions in Flatland

- create environments and trains with malfunctions
- gilkra2025a dealt with train malfunctions (secondary encoding)
- waldow2025

2.3 Track Malfunctions

Track malfunctions are a common real world issue in railway systems. It negatively affects punctuality and interferes with scheduling of trains. Deutsche Bahn reports old facilities in need of repairs just as well as an intense, high-frequent use of existing infrastructure as reasons for malfunctions around the track [2]. Other factors can be weather incidents, such as flooding [4] or fires [2], or trespassing individuals that make railway sections temporarily unusable [1].

[3] If EquinOCS, our proceedings submission system, is used, then the disclaimer can be provided directly in the system.

3 Our Code

Building on existing work about malfunctioning trains, we implement track malfunctions to simulate realistic scheduling challenges (see Sections 2.2 and 2.3). We base our approach on previous work by gilkra and want to build a similar structure for dealing with interruptions in the original schedule. Flatland doesn't natively include track malfunctions. Alternatively, we have included the generation of malfunctions in the main solving pipeline (see Section 3.3). By reusing plans for trains from the original schedule, we aimed to improve computation time during rescheduling through a secondary encoding (see Section 3.4).

3.1 Python Part

Track Malfunction Implementation Since we could not implement track malfunctions directly in Flatland, we implemented them in Python code that runs Clingo with Flatland input.

The added code consists of multiple parts:

- Creating track malfunctions either randomly or deterministically
- Translating the track malfunction Python object into Clingo
- Forwarding the atoms to Clingo
- Logging the malfunction in a file
- Depicting the malfunction in the Flatland output

How is a track malfunction created? A track malfunction Python object is a tuple consisting of three components:

```
(cell, timestep, duration)
```

The `cell` is an `(X, Y)` tuple that defines where the track malfunction occurred.

The `timestep` represents the timestep at which the track malfunction was triggered. During this timestep, the malfunction has not yet occurred; it starts in the following timestep. This was an architectural decision to ensure that trains cannot be on a malfunctioning track when the malfunction starts.

The `duration` specifies the number of timesteps for which the track malfunction occurred.

Track malfunctions can either be created randomly or predefined by the user. The variable `predefined_malfunction` is used to store malfunctions that the user wants to enforce on the environment. If the variable is empty, there is a 1% chance of a malfunction occurring at every timestep (this probability can be adjusted). In the current setup, only one malfunction can occur per timestep. **(Improvement idea)**

How is a malfunction implemented in Clingo? When a malfunction is triggered in Python, multiple atoms and rules are generated and forwarded to the program that calculates the rescheduling.

- `action` **constraint rules** – To preserve the actions taken up to this point, a constraint rule is generated for each action, ensuring that all past actions must be present in the output of the rescheduling.
- `track_malfunction` **atoms** – Used to provide the timestep and duration of the malfunction.
- `position` **constraint rules** – Since the track malfunction starts one timestep after it is triggered, the actions at the time the malfunction is triggered are preserved. Consequently, their positions in the next timestep are also preserved. These constraints only include the next position.
- `planned_action` **atoms** – Include all actions that were planned before the malfunction occurred.
- `planned_position` **atoms** – Include all positions that were planned before the malfunction occurred.
- `current_position` **atoms** – Include all positions occupied at the timestep at which the malfunction occurred.

How is the malfunction logged? The malfunction is logged in a CSV file with the following parameters:

```
timestep;cell;duration
```

How is the malfunction depicted? Similar to a train malfunction, a track malfunction is depicted as a red **X** over the malfunctioning track.

3.2 base encoding

connections-nswe

generators

constraints

3.3 Additions to solve.py

3.4 Secondary Encoding

No consequence

Simple Consequence (no collision)

Complex Consequence (collision/ multi-agent rescheduling)

create accidents

4 Experiments

4.1 with and without heuristic

4.2 compare grounding for two versions of trains can't be in same cell at same time rule

5 Log / Timeline

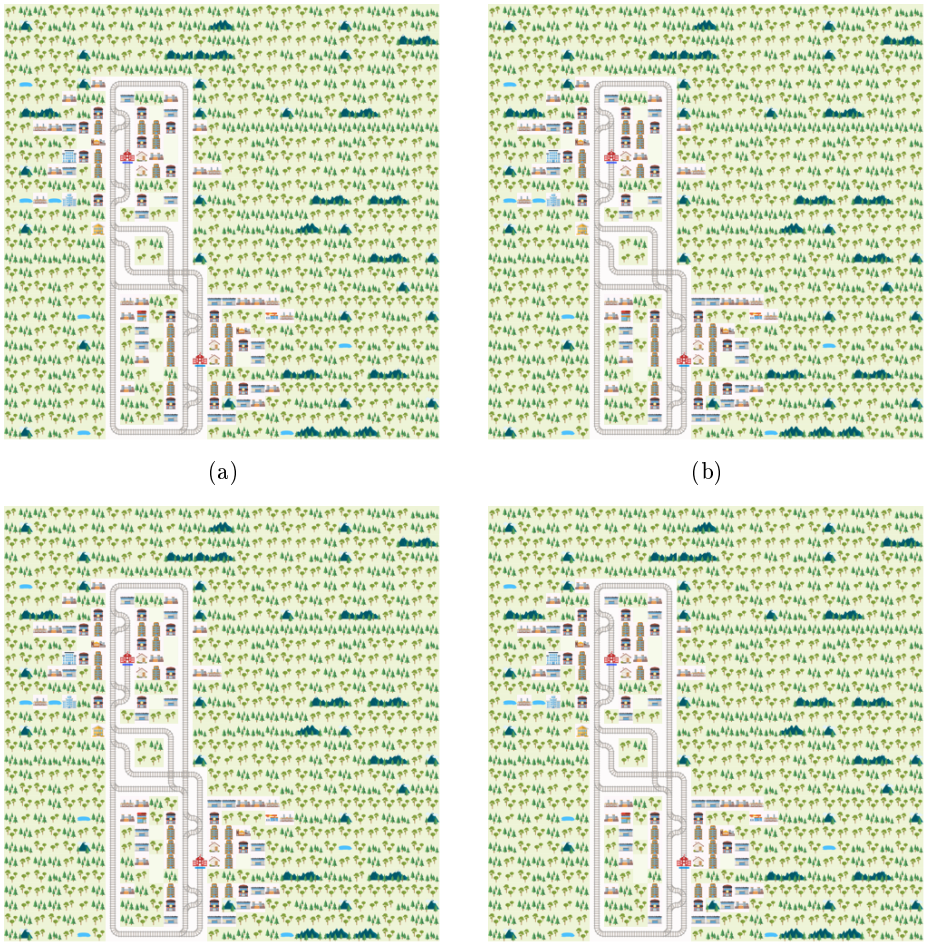
- merge encodings (May 29)
- literature review zank2026, gilkra2025a, waldow2025a (June 6)
- fix visualization issue aka add wait before start time (June 9)
- "crack the egg" pkl-file (June 17)
- understand solve.py (June 18)
- create pipeline for track malfunction in parallel to train malfunction TrackMalManager etc. (June 22)
- pick relevant track for malfunction randomly (July 2)
- create visualization for track malfunction ()
- create track malfunction log (July 13)
- start secondary encoding (no consequence or rescheduling necessary) (July 14)
- expand secondary encoding: handle rescheduling of directly affected train (August 2)
- finish secondary encoding: handle collisions due to rescheduling (August 3)
- improve encodings: minimize grounding, remove unnecessary rules (August 7)
- add heuristic, code cleaning and remove can_dir (August 10)
- separate predefined and random track malfunctions, code clean-up (August 13)

6 Future Work

- explore options instead of random malfunctions (frequented tracks, crosses, etc.)
- explore option of tertiary encoding (recalculate from scratch if malfunction hits early on: first > secondary > first)
- heuristic assumes loose rail network BUT not realistic (maybe discussion part)
- handle unsatisfiable as a result of calculating after malfunction (?)

7 Experiment 1

We compared the use of the secondary encoding with rescheduling the entire environment anew with the base encoding. For this, we have created four testing environments of varying sizes (2 small and 2 medium environments - see Figure 2). In each environment we have identified tracks for all three types of track malfunctions (simple, complex, no consequence - see Section 3.4). We ran each environment and malfunction in two modes. Mode 1-1 scheduled and rescheduled with the base encoding. 1-2 used the base encoding only for scheduling and the secondary encoding for rescheduling.



(a) (b)

(c) (d)

Fig. 2: These are the testing environments.

Env name	Mal type	Mode	Schedule	Reschedule
env_016	simple	1-2	0.092	0.084
	simple	1-1	0.095	0.082
	complex	1-2	0.097	0.070
	complex	1-1	0.092	0.089
	no cons	1-2	0.185	0.116
	no cons	1-1	0.100	0.107
env_028	simple	1-2	0.114	0.038
	simple	1-1	0.057	0.047
	complex	1-2	0.057	0.056
	complex	1-1	0.057	0.055
	no cons	1-2	0.065	0.050
	no cons	1-1	0.065	0.061
env_033	simple	1-2	0.651	0.389
	simple	1-1	0.583	0.607
	complex	1-2	0.589	0.867
	complex	1-1	0.598	3.747
	no cons	1-2		
	no cons	1-1		
env_035	simple	1-2	26.135	2.116
	simple	1-1	26.240	11.449
	complex	1-2	26.299	2.053
	complex	1-1	26.251	35.805
	no cons	1-2	26.471	6.579
	no cons	1-1	26.219	22.771

1-1: base encoding for scheduling and rescheduling
1-2: base encoding for scheduling, secondary encoding for rescheduling

Table 2: Results for testing environments with different malfunction types.

References

1. Bahn-Verspätungen durch Menschen an und auf Gleisen. Süddeutsche Zeitung (2023), https://www.sueddeutsche.de/wirtschaft/statistik-bahn-verspaetungen-durch-menschen-an-und-auf-gleisen-dpa.urn-newsml-dpa-com-20090101-230610-99-05514, Accessed: 18.08.2026
2. Integrierter Zwischenbericht 2025. Deutsche Bahn AG (2025), https://zbir.deutschebahn.com/2025/de/konzern-zwischenlagebericht-ungeprueft/qualitaet-und-sicherheit/puenktlichkeit/, Accessed: 10.08.2026
3. Gebser, M., Kaminski, R., Kaufmann, B., Schaub, T.: Multi-shot asp solving with clingo (2018), https://arxiv.org/abs/1705.09811
4. Loderbauer, G.: Punctuality in rail transport in Autria in 2024. Agentur für Passagier- und Fahrgastrechte (2025), https://en.apf.gv.at/news-detail/P%C3%BCnktlichkeit2024, Accessed: 18.08.2026

5. Mohanty, S., Nygren, E., Laurent, F., Schneider, M., Scheller, C., Bhattacharya, N., Watson, J., Egli, A., Eichenberger, C., Baumberger, C., Vienken, G., Sturm, I., Sartoretti, G., Spigler, G.: Flatland-rl: Multi-agent reinforcement learning on trains (2020), https://arxiv.org/abs/2012.05893
6. Murphy, K.R.: Flatland (2026), https://github.com/krr-up/flatland, Accessed: 17.08.2026